JobTracker Hub

Item 6 — Development Log & Checkpoint History

What this document is. A running log of the Item 6 (Application Dossier) feature work, written so a new chat session – with no memory of previous ones – can pick up exactly where the last one stopped. Each checkpoint section below corresponds to one delivered, tested zip. This is a working document, not the end-user manual (see `JobTracker_User_Guide.pdf` for that).

Feature	Item 6 — Application Dossier / Application Intelligence
Workflow	One chat = one tested, zipped checkpoint. Claude does not touch GitHub or build the DMG.
Base project	JobTracker Hub v1.1.0, 48/48 tests passing before Item 6 began.
This log	Updated at the end of every checkpoint chat; latest checkpoint's "Next Steps" box is always the first thing to read.

Repository: https://github.com/willmaddock/jobtracker-hub
Log version: Checkpoint 6 | **Updated:** August 29, 2026

Hand this PDF (or its `.tex` source) plus the latest checkpoint zip to the next chat as its starting context.

Contents

1 How To Use This Document

For the human. At the start of a new chat, upload the latest checkpoint zip and this `.tex` (or the compiled PDF). Tell the new Claude session to read the Next Steps box for the most recent checkpoint before doing anything else.

For the next Claude session. This log is cumulative – do not delete earlier checkpoint sections when you add a new one. Append a new `Checkpoint N` section below the most recent one, update the Next Steps box to reflect the new state, and bump the cover page's checkpoint number and date. Keep every checkpoint's original "What was done" section intact as history, even after its follow-up items are completed elsewhere.

1.1 Ground rules that apply to every checkpoint

- Claude works only on the local project source. No git commit, no git push, no GitHub Releases, no building the `.dmg`.
- The human integrates each checkpoint zip into their real working repository, handles GitHub, builds the DMG, and performs real Safari/packaged-app validation.
- Every checkpoint must leave the existing test suite passing. Existing tests are never weakened or removed to make new tests pass.
- A checkpoint is produced only when the working tree has reached a coherent, testable state – never a zip of broken work just because the chat is ending.
- Extraction and inference stay deterministic and local (regex/heuristics), never a required cloud/AI dependency, per the project's local-first privacy posture.
- **Keep this log from growing without bound.** Once more than 3 checkpoints exist, condense every checkpoint older than the most recent 3 down to a short paragraph (scope + one-line outcome + deliverable filename) instead of its full "What was done" detail. Never condense the *Next Steps* section, and never condense any of the 3 most recent checkpoints.

2 Next Steps

Read this first. This box always reflects the state after the *most recently completed* checkpoint. If you are starting a new chat, this is your starting instruction.

As of Checkpoint 6 (Auto-Fill Date Applied From Confirmation Evidence), the recommended next task is:

1. **Run the real `pytest` suite and do a manual click-through** of Checkpoint 6 against the real working tracker – this session had no network access to install `pytest`/`fastapi`/`httpx` or `@babel/core`, so the new backend tests were only verified by direct module-level simulation and the frontend edit only by diff/bracket-balance review, not an actual test run or a real render. See §6.3 for exactly what still needs confirming.
2. **Timeline view**, built from document mtimes/doc_types per application (drafted → applied → interviewing/rejected), using the same `effective\_doc\_type` data the dossier and status-inference logic already read.
3. The archive/lifecycle engine remains deliberately deferred (see Checkpoint 1's design discussion) until the above is validated against the real 119-application dataset.

Checkpoint 6 (this checkpoint) still needs its post-checkpoint verification pass. Unlike Checkpoints 3 and 5, this checkpoint's own session had no network access to install test/build

tooling, so its correctness rests on direct code-level simulation and structural review rather than a real `pytest` run or a real JSX render/click. See §6.3 for the specifics – closing that gap is item 1 above.

Not yet started (later checkpoints, per the agreed roadmap): everything in the items above, plus any follow-up on the known Dossier limitations noted in §3.3/§4/§5/§6 (trailing-boilerplate bleed into the last role section; arbitrary-but-deterministic alphabetical tiebreak when a folder has more than one `job\_posting`-classified document; no per-document source attribution on merged contacts; date-applied detection's US-slash-date and English-month-name-only patterns; no signal for which confirmation document wins when a folder has more than one).

3 Earlier Checkpoints (condensed)

Per this log's own ground rules (§1), checkpoints older than the most recent three are condensed to scope + outcome + deliverable. Full detail for Checkpoints 4–6 follows below.

3.1 Checkpoint 1 — Extraction Foundation

Built `_app/extract.py`: deterministic text extraction from `.pdf`/`.txt` plus conservative regex-based email/phone/URL extraction, backed by a content-hash-keyed cache in `overrides_store.py` (`document_extractions` table). 67 passed, 1 warning (19 new tests). Deliverable: `jobtracker-hub-item6-check`

3.2 Checkpoint 2 — Job-Posting Section Extraction

Built `_app/role_extract.py`: header-pattern splitting of extracted text into role_summary/duties/required_qua with header phrasing collected empirically from 20 real employers' postings in the working tracker. Runs unconditionally on all text (never gated on doc_type, to preserve the content-hash cache's path-independence). `EXTRACTOR_VERSION` bumped to `"2"`. 82 passed, 1 warning (15 new/modified tests), plus a real-corpus sanity pass against all 39 job-posting-classified PDFs in the working tracker. Deliverable: `jobtracker-hub-item6-checkpoint02.zip`.

3.3 Checkpoint 3 — Application Dossier Backend/API Assembly

First API wiring for Item 6: new `GET /api/applications/\{item\_id\}/dossier`, assembling one per-application payload from `dossier.py`'s new `assemble_dossier()` – merged contacts across every document in the folder, plus the four Checkpoint-2 role sections from whichever document is picked as the job posting (`effective_doc_type == "job_posting"`, alphabetical tiebreak when more than one candidate exists, listed in `job_posting_candidates` for visibility). A document that fails to extract is skipped and reported in `extraction_errors` rather than failing the whole Dossier. 82 passed unaffected, plus 4 new tests in `test_dossier.py` (verified via a direct-simulation harness against real extraction code, since that session had no network access to run `pytest` itself). Deliverable: `jobtracker-hub-item6-checkpoint03.zip`.

4 Checkpoint 4 — Application Dossier Frontend UI

4.1 Scope

Surface Checkpoint 3's `GET /api/applications/\{item\_id\}/dossier` endpoint in the frontend, per Checkpoint 3's Next Steps box: a Dossier panel on the application detail view showing the four role sections and merged contacts, with clear "Not detected"/empty states (never blank or silently hidden). Also closed a small gap this surfaced: the Contacts section's email/phone action buttons needed a way to open `mailto:`/`tel:` links through the app's existing OS-opener path, so `api.py`'s `/api/open-url` scheme allowlist was extended to match.

4.2 What was done

- `_app/frontend/index.html`:
  - New components `DossierSection` and `ContactGroup` (see §4.3 below), plus `ROLE_SECTION_DEFS`/`NOT_DETE` constants mirroring `role_extract.py`'s `SECTION_KEYS`/`NOT_DETECTED` on the Python side.

  - New `.dossier-*`/`.contact-*` CSS rules reusing the app's existing design tokens (`-panel`, `-border`, `-text-soft`, the `.card`/`.section-label` conventions) instead of introducing a new visual language.
  - `DetailPane` renders `<DossierSection>` above "Attached Documents", and gained two new props (`dossier`, `fireToast`).
  - `PipelinePage` gained `dossierByItemId`/`ensureDossier` props, fetching the selected application's Dossier in the same effect that already fetches its documents.
  - `App()` gained a `dossierByItemId` cache with `ensureDossier` (fetch-once, cache) and `invalidateDossier` (evict-on-mutation) – deliberately mirroring `docsByItemId`'s existing shape rather than inventing a second caching pattern. See §4.4 for why eviction, not in-place patching, was the right call here.

- `_app/api.py`: `/api/open-url`'s scheme check refactored into an `_OPEN_URL_SCHEMES` tuple and extended from `http://`/`https://`-only to also accept `mailto:` and `tel:`, so `ContactGroup`'s action buttons can route through the same OS-opener endpoint Search Hub already uses for external links, instead of a raw `window.location.href` (unreliable inside the packaged webview – see `openExternalUrl`'s own comment in the frontend for the original reasoning, which now applies equally to mailto/tel).

- `tests/test_open_url.py` (new): 12 tests against the scheme allowlist – every accepted scheme returns `200` (including a `mailto:` with query params), every rejected scheme (`javascript:`, `file:`, `ftp:`, empty/whitespace/non-URL strings) returns `400` naming all four allowed schemes, whitespace is trimmed before the check, and the OS-opener call (`subprocess.run`/`os.startfile`, monkeypatched) is never reached on a rejected request.

4.3 Component design: DossierSection / ContactGroup

`DossierSection` takes the already-loaded `app` object (for the context row: company, job title, effective status, date applied, days since activity – all already present on every application row, no second fetch needed) plus the `dossier` object from the new cache. Three states: `dossier === undefined` renders a loading placeholder (fetch in flight); `dossier` falsy after a failed fetch renders nothing (fails soft, doesn't break the rest of `DetailPane`); the loaded object renders the context row, a note naming which attached document supplied the role sections (with a "View" button into the existing inline document viewer, when that document is present in the currently-loaded doc list), the four role-section blocks, and the Contacts block. Each role-section block shows `NOT_DETECTED` in muted italic rather than an empty string when absent, matching the backend's own never-fabricate convention end-to-end. `ContactGroup` renders one column per contact kind (emails/phones/links); each value gets a copy button (`navigator.clipboard`, falling back to a toast telling the user to copy manually on failure – the same fallback pattern `index.html` already uses for `SettingsModal`'s diagnostic-report copy button) and an action button that opens `mailto:`/`tel:`/`https:` via `openExternalUrl`.

4.4 Design decision: cache-evict, not cache-patch, on mutation

`docsByItemId` is patched in place from each mutation's response, because that response already *is* the new per-item document list. The Dossier's response is a derived aggregate – which single document counts as "the" job posting (via the alphabetical tiebreak, §3.3), and the deduped union of contacts across every document in the folder – that a single mutation's response can't fully express client-side (e.g. deleting the current job-posting document should promote the next `job_posting_candidates` entry, if any, but `performDelete`'s response has no way to say what that is). `invalidateDossier` evicts the cache entry on all four document-mutation paths (`performDelete`, `renameDocument`, `uploadDocuments`, `saveDocTypeOverride`) and lets `ensureDossier` do one clean re-fetch next time the item is selected, rather than attempting to patch a shape that doesn't map onto any single mutation's response. The cost is one extra round-trip per mutation, judged acceptable since Dossier-affecting mutations (uploads/renames/deletes/fix-type) are deliberate, occasional

actions, not a hot path.

4.5 Test results

```
$ python3 -m pytest -q
100 passed, 1 warning
```

Unlike Checkpoints 2–3, this session had network access to install `_app/requirements.txt` + `requirements-dev.txt` and actually execute the full suite, rather than falling back to manual verification. All 88 pre-existing tests pass unchanged (including Checkpoint 3's own 4 dossier tests, run for real this time – they'd only been verified by a standalone manual harness before), plus the 12 new `test_open_url.py` tests. The frontend JSX was additionally checked with a real parser – `@babel/core` + `@babel/preset-react`'s `transformSync` against the extracted `<script type="text/babel">` block, installed via `npm` in a scratch directory – rather than only a bracket-balance count (which the two prior sessions had to rely on, for lack of network access); a real parser catches malformed JSX (mismatched tags, bad prop syntax) that balanced brackets alone can miss. Parses clean.

Not covered by any automated test: this repository has no frontend component/JS test harness (backend-only `pytest`; confirmed by scanning for `.test.js`/Playwright/Selenium – none exist), so `DossierSection`/`ContactGroup`'s actual rendering and the copy/mailto/tel button behavior have only been verified by the real JSX parse plus manual re-reading of every changed call site's props against its consumer's destructured signature – not an actual render or click. **A manual click-through is recommended before treating this checkpoint as fully verified** – see the Next Steps box.

4.6 Post-checkpoint fix: Dossier not refreshing after "Fix type"

The manual click-through this section recommended was carried out in a later chat and surfaced one real bug missed by both the JSX parse and the prop-signature review above (neither exercises actual React re-render behavior). Uploading a new document or renaming one correctly refreshed the Dossier with no page reload. Using "Fix type" to reclassify a document (e.g. unclassified → `job_posting`) while staying on the *same* application did not – the card stayed on "Loading dossier…" until the page was manually reloaded.

Root cause: `PipelinePage`'s data-fetch effect was

```
useEffect(() => {
  if (selected) { ensureDocs(selected.id); ensureDossier(selected.id); }
}, [selected && selected.id]);
```

– dependent only on *which* application is selected, not on whether that application's cached dossier is still present. `invalidateDossier` (§4.4) correctly evicts `dossierByItemId[itemId]` on `saveDocTypeOverride`, but with `selected.id` unchanged (the user never navigated away), nothing else was left to call `ensureDossier` again for that same id – the cache entry just stayed evicted (`undefined`), which is exactly the state `DossierSection` renders as its loading placeholder. A full reload happened to fix it only because it resets all React state and re-runs every effect from scratch.

Fix: widen the effect's dependency array to also track the selected item's current cache value, not just its id:

```
useEffect(() => {
  if (selected) { ensureDocs(selected.id); ensureDossier(selected.id); }
}, [selected && selected.id, selected && dossierByItemId[selected.id]]);
```

`PipelinePage` already received `dossierByItemId` as a prop, so no new plumbing was needed. `ensureDocs`/`ensureDossier` are both already no-ops once their target is cached (see their own `prev[itemId] !== undefined` / `prev[itemId]` checks), so the wider dependency does not cause repeated fetching once a value resolves – it only re-fires the fetch on the specific transition that matters: cached value → evicted.

Verified (real click-through, not static analysis this time): re-tested rename, new-file upload, and "Fix type" against the running app after the fix. All three now update the Dossier in place with no manual page refresh required. This closes the one open item from §4's original verification caveat – Checkpoint 4 is now considered fully verified by direct use, not just by parse/read-through.

4.7 Deliverable

`jobtracker-hub-item6-checkpoint04.zip` – full repository source (minus `.venv`, `.git`, caches, and local workspace state), plus an updated `CHANGES.md` at the repository root.

4.8 Known limitations

- No date-applied evidence suggestion yet – confirmed there is still no backend signal for this anywhere (`dossier.py`, `extract.py`, `db.py`), so building it is real detection work deferred to the next checkpoint, not something that could be faked in this pass.
- No "confirmed" evidence state for role sections or contacts – only "detected"/`NOT_DETECTED` exist in the real data; a "confirmed" state would need a manual-override mechanism that doesn't exist in the backend yet.
- Carries forward all Checkpoint 1–3 limitations unchanged (US-format phone regex, empty-password-only PDF decryption, no `.docx` support, trailing-boilerplate bleed into the last-matched role section, arbitrary-but-deterministic alphabetical job-posting tiebreak, no per-document source attribution on merged contacts).

5 Checkpoint 5 — Date-Applied Evidence Suggestion

5.1 Scope

The Next Steps item Checkpoint 4 flagged as real detection work, not UI wiring: detect an application date from a document's own text and surface it in the Dossier as an explicit accept/reject suggestion next to the existing Date Applied field – never a silent overwrite of a manually-set `date_applied` override.

5.2 What was done

- **New module `_app/date_extract.py`:**
  - `extract_application_date(text)` – returns an ISO `YYYY-MM-DD` string or `None`. Never fabricates, same convention as `role_extract.py`.
  - Primary pattern: the mail client's own `Date: July 10, 2025 at 10:56 AM` header line that "print to PDF" confirmation emails carry – the single most reliable signal available, since it's the mail system's timestamp, not a guess from body prose. Handles an abbreviated month (`Jul 30, 2025`) and a repeated `Date:` label (`Date: Date: Date: Jul 30, 2025`), both observed in the real working tracker.
  - Fallback pattern: a keyworded numeric date (`received`/`submitted`/`applied` within 120 characters, never crossing a sentence boundary, of a `MM/DD/YYYY`) for ATS confirmation bodies (schooljobs.com, governmentjobs.com) that restate the timestamp inline instead of via an email header.
  - Both patterns were collected empirically against real confirmation-email PDFs in `working-db.zip` (Extend, FRCC, Metro Water Recovery, Vantor, MAXAR, T-Mobile, ...) rather than guessed

abstractly – a spot-check across 461 PDFs under `Applications/` found the email-header pattern present in 104 of them.

  - An invalid calendar date from a false-positive-shaped match (e.g. a corrupted "February 32, 2025") is rejected via Python's own `date()` validation rather than coerced into something plausible-looking.

- **`_app/extract.py`:** `EXTRACTOR_VERSION` bumped `"2"` → `"3"`. `extract_document()` now merges a `detected_date_applied` field into its result dict (via `date_extract.extract_application_date()`, or `None` on extraction failure), following the same "runs on ALL text, not gated on doc_type" rule Checkpoint 2 established for role sections – gating it would break the `overrides.db` extraction cache's content-hash path-independence guarantee.
- **`_app/dossier.py`:** new `DATE_EVIDENCE_DOC_TYPES = ("application_confirmation", "job_posting")` priority tuple. `assemble_dossier()` now collects each document's `detected_date_applied` alongside the existing job-posting-pick loop, and picks the alphabetically-first relpath within the highest-priority doc type that had any hit – see §5.2 for why confirmation beats posting. Adds `detected_date_applied` / `detected_date_source_relpath` (both `None` when nothing was found) to the returned dict. Never writes to `date_applied` itself.
- **`_app/api.py`:** `/api/applications/\{item\_id\}/dossier`'s docstring updated to describe the two new response fields; no behavioral or schema changes (the endpoint already returns `dossier.assemble_dossier()`'s dict as-is).
- **`_app/frontend/index.html`:** `DossierSection` gained a date-suggestion block under the existing "Date applied" context field – shown only when `detected_date_applied` is present and differs from the application's current `date_applied`, with "Use detected date" / "Keep existing" buttons and a new `.dossier-date-suggestion` CSS rule reusing existing design tokens. `DossierSection` took a new `onUseDetectedDate` prop instead of the raw `onSaveOverride`, so the actual save logic (and the local `dateApplied` state the editable form field further down reads) stays owned by `DetailPane` – see §5.2 for why. `DetailPane` gained a `useDetectedDate(iso)` handler mirroring the existing `quickFollowUp`/`quickResetActivity` pattern, and passes it down.
- **Tests:** `tests/test_date_extract.py` (new, 8 tests) covering both patterns, the abbreviated-month/repeated-label variants, the 120-char/sentence-boundary guard against false positives, no-pattern and empty-text cases, and invalid-calendar-date rejection. `tests/test_dossier.py` extended with 3 new tests: confirmation-email date detected end-to-end through the real API, confirmation prioritized over a job posting's own date when both are present, and the `None`/`None` case when no document yields a date.

Design note – why `application_confirmation` beats `job_posting` for date evidence. A confirmation email's own header timestamp is a direct record of when the application was submitted. A job posting's text at best names when the position was *listed* – a different, earlier date that would misrepresent `date_applied` if surfaced with equal priority. `DATE_EVIDENCE_DOC_TYPES` therefore checks `application_confirmation` documents first and only falls back to `job_posting` text when no confirmation email is present in the folder (or none of them yielded a recognizable date). This mirrors Checkpoint 3's job-posting-pick precedent: the choice is deterministic and made at the assembly layer, never inside `extract.py`.

Design note – keeping the suggestion state in `DetailPane`, not `DossierSection`. `DetailPane` already owns a local `dateApplied` state that the editable Date Applied form field (further down the same pane) reads and writes, synced from `app.date_applied` only when `app.id` changes (an existing pattern predating this checkpoint). Had `DossierSection` called `onSaveOverride` directly on "Use detected date", the context row at the top of the Dossier card would update immediately (it reads `app.date_applied` directly) while the editable field lower in the same pane would silently keep showing the pre-suggestion value until the application was reselected – two fields disagreeing about the same underlying value in the same screen. Routing the click through a `useDetectedDate` handler defined in `DetailPane` itself (mirroring `quickFollowUp`/`quickResetActivity`, which already face this exact same local-state-vs-override tension) keeps both fields in sync from a single write.

5.3 Test results

```
$ python3 -m pytest -q
111 passed, 1 warning
```

All 100 pre-existing tests pass unchanged, plus 11 new (8 in `test_date_extract.py`, 3 in `test_dossier.py`). The frontend JSX was checked the same way Checkpoint 4 established – `@babel/core` + `@babel/preset-react`'s `transformSync` against the extracted `<script type="text/babel">` block – and parses clean. That parse check caught a real issue during this checkpoint's own development, not after the fact: an initial draft called `useState`/`useEffect` inside `DossierSection` *after* its two existing early returns (`dossier === undefined` / `!dossier`), which violates React's Rules of Hooks – the number of hooks called would differ between the loading/null renders and the loaded render, which React detects at runtime (not something a JSX parse alone would catch, since the code is syntactically valid either way; this was caught by reviewing the diff against React's hook-ordering rule before it reached a click-through). Fixed by moving both hooks to the top of the component, before either early return, so they run unconditionally on every render regardless of which branch a given render takes.

Not covered by any automated test at the time this checkpoint was cut: same standing limitation as every checkpoint touching the frontend (no JS/component test harness in this repository) – the actual click behavior of "Use detected date"/"Keep existing" and the interaction between `DossierSection`'s suggestion-dismissal state and `DetailPane`'s form field had only been verified by the JSX parse plus manual re-reading of props/hook-ordering, not an actual render or click. A manual click-through was recommended before treating this checkpoint as fully verified – carried out in a later session, see §5.4.

5.4 Post-checkpoint verification: manual click-through against the real tracker

The manual click-through this checkpoint originally recommended was carried out in a later chat, directly against the real 110-application working tracker (`python3 -m venv .venv`, install, `uvicorn api:app -reload`) rather than synthetic fixtures. **No bugs found** – every case exercised behaved as designed:

- **Suggestion renders and both fields stay in sync.** For an application with `date_applied` unset (Adams County), the Dossier showed `Detected 2026-02-17 from Application received by Adams County.pdf` with "Use detected date"/"Keep existing". Clicking "Use detected date" updated the context row's Date Applied *and* the Days Since Activity figure (0d → 193d, since `db.py`'s existing staleness logic falls back to `date_applied` once it's set) in the same action – exactly the two-fields-in-sync behavior §5.2's design note was written to guarantee.
- **Suggestion box disappears once accepted.** After "Use detected date" set `date_applied` to match `detected_date_applied`, the suggestion box correctly stopped rendering on the next

view of that application (AffiniPay, FRCC) – confirming `showDateSuggestion`'s equality check against the now-current value, not a hard-coded "already suggested once" flag.

- **Date detection is independent of role-section/job-posting availability.** An application folder with no document classified as `job_posting` (FRCC 2 – IT Manager) correctly showed the existing "No document in this application's folder is marked as a job posting..." message for the four role sections *alongside* a working date-applied suggestion (`Detected 2026-08-02 from Application Received.pdf`) – confirming `DATE_EVIDENCE_DOC_TYPES` and the job-posting pick are genuinely independent code paths, as designed.
- **Downstream staleness/Needs-Attention logic picks up the accepted date correctly.** After accepting FRCC 2's detected date, the item correctly flagged "27d since date applied • needs attention" and appeared on the Needs Attention page alongside AffiniPay (339 days quiet) – confirming the Checkpoint-1-era `reference_date = activity_override or date_applied or last_activity` logic in `db.py` picks up a Dossier-suggested date exactly the same as a manually-typed one, with no special-casing needed.
- **Contacts rendered correctly alongside the date suggestion** on the same card (multiple emails, a phone number, working Email/Call buttons) – unaffected by this checkpoint's changes, confirming no regression to Checkpoint 3–4's Contacts block.

Also confirmed in this session: the full suite (`111 passed, 1 warning`) was independently re-run against a clean install on a second machine/environment (macOS, Python 3.14.7, pytest 9.1.1 – vs. this checkpoint's own Python 3.12.3 run), giving the same 111/111 result and confirming `date_extract.py`'s patterns aren't accidentally dependent on a specific Python or `pypdf` version.

This closes the one open item from §5's original verification caveat – Checkpoint 5 is now considered fully verified by direct use, not just by parse/read-through, matching the bar Checkpoint 4 set in §4.6.

5.5 Deliverable

`jobtracker-hub-item6-checkpoint05.zip` – full repository source (minus `.venv`, `.git`, caches, and local workspace state), plus an updated `CHANGES.md` at the repository root.

5.6 Known limitations

- English month names and US-style `MM/DD/YYYY` only – no ISO (`YYYY-MM-DD`) body-text pattern, no other locales' date formats. Not seen in the real working tracker's corpus (all US-based applications), so not built speculatively; would need real examples before adding.
- The keyworded fallback pattern is scoped to `received`/`submitted`/`applied` only – a confirmation email phrased entirely differently (no header `Date:` line and none of those three verbs near its date) reports `None` rather than a guess, by design, but worth knowing going in.
- No signal for *which* `application_confirmation` document's date is most trustworthy when a folder has more than one (e.g. an initial confirmation plus a later status-update email that also matches the confirmation classifier) – same alphabetical-tiebreak posture as the existing job-posting pick, not a new gap this checkpoint introduces, but worth revisiting together if it turns out to matter.
- Carries forward all Checkpoint 1–4 limitations unchanged (US-format phone regex, empty-password-only PDF decryption, no `.docx` support, trailing-boilerplate bleed into the last-matched role section, arbitrary-but-deterministic alphabetical job-posting tiebreak, no per-document source attribution on merged contacts, no "confirmed" evidence state for role sections/contacts).

6 Checkpoint 6 — Auto-Fill Date Applied From Confirmation Evidence

6.1 Scope

A review of Checkpoint 5 pointed out that "detection could be wrong" and "detection could destroy something" are two different risks that Checkpoint 5's design had conflated: when `date_applied` is already blank, a wrong auto-fill is a data-quality annoyance fixable later, not a silent loss of something the user deliberately typed – so requiring a click in that specific case buys no real safety, only friction. This checkpoint splits the Checkpoint 5 suggestion by evidence tier (confirmation vs. posting, `DATE_EVIDENCE_DOC_TYPES`'s existing two-entry priority order) and by whether `date_applied` already has a value, and auto-fills the one case that's actually safe: field empty, confirmation-tier evidence found.

6.2 What was done

- `_app/dossier.py`: `assemble_dossier()` now also returns `detected_date_evidence_tier` (`"confirmation"`, `"posting"`, or `None`) – a direct read of which entry in `DATE_EVIDENCE_DOC_TYPES` won, so callers don't have to re-derive it. Still never writes to `date_applied` itself.
- `_app/overrides_store.py`: new `date_applied_source` column on `item_overrides`, with a migration for existing `overrides.db` files (same `PRAGMA table_info` pattern the `activity_override` migration already established) – `"confirmation"`/`"posting"` when the current `date_applied` came from an accepted or auto-filled detection, `NULL` when the user typed it. Display-only provenance; never affects which date wins on a conflict. `upsert_override()` threads it through the existing merge-from-existing-row pattern.
- `_app/api.py`: `OverrideRequest` gained `date_applied_source`; `save_override()` clears it automatically whenever `date_applied` is written without a source also being sent (a manual retype makes any earlier provenance label stale), and applies it when the caller does send one. `/api/applications/\{item\_id\}/dossier` now performs the auto-fill itself: when `date_applied` is unset *and* the winning evidence is confirmation-tier, it writes `date_applied` + `date_applied_source` before returning, and reports `date_applied_auto_filled: true` in the response. Posting-tier evidence never auto-fills. A `date_applied` that already has any value – typed, or auto-filled by an earlier call to this same endpoint – is never touched again, regardless of what's detected afterward. See §6.2 for why that last rule matters concretely.
- `_app/frontend/index.html`: `DossierSection`'s date-applied area now renders one of three mutually exclusive things: (1) a **conflict** prompt (value already on record, newly detected evidence disagrees) – the same two-button "Use detected date"/"Keep existing" UI as Checkpoint 5, applying regardless of evidence tier; (2) a **weak suggestion** (field empty, only posting-tier evidence) – new single "Use this date" button, worded as a review prompt, new `.dossier-date-suggestion-weak` CSS rule; (3) a **provenance caption** (value present, came from a confirmation-tier auto-fill/accept, nothing currently conflicting) – new quiet `.dossier-date-provenance` line, no buttons. `DetailPane`'s `useDetectedDate(iso, source)` handler now forwards the evidence tier as `date_applied_source`. `ensureDossier` now calls `refreshApps()` whenever a dossier response reports `date_applied_auto_filled: true`, since the Pipeline list (not the dossier response) is what feeds `date_applied`/`date_applied_source` to the context row and the Needs-Attention staleness clock.
- **Tests:** `tests/test_dossier.py` extended with a 7-test Checkpoint 6 section (empty+confirmation auto-fills; empty+posting-only stays unset and suggests; existing value + differing confirmation preserved; existing value matching detection produces no conflict signal; an auto-filled value + a later differing detection still preserved; confirmation still wins priority and still auto-fills; no detectable date never produces a false auto-fill flag). New `tests/test_overrides_store.py` (5 tests) – direct unit tests of the migration and of `upsert_override`'s round-trip/clear/preserve behavior for `date_applied_source`.

Design note – why evidence tiers, not a confidence score. The originating review considered a three-tier confidence model, but `date_extract.py` doesn't score confidence within a match – it's binary (the pattern matched, or it didn't). The real distinguishing signal already in the code is *which document type* the date came from, which `DATE_EVIDENCE_DOC_TYPES`'s existing two-entry priority order already captures, so this checkpoint reframes the suggestion around that existing split rather than inventing a scoring mechanism the extractor doesn't have. A posting-derived date is still surfaced (never discarded outright) when no confirmation email exists – it's just never auto-applied.

Design note – auto-fill can only ever fire once, into a genuinely empty field. Once a value lands in `date_applied` – however it got there – later detection results go through the same conflict-suggestion path as a manually-typed value, never a silent second overwrite. This matters concretely for a duplicate confirmation email with a different timestamp (e.g. a resend): without this rule, the *second* dossier fetch after that email appears would silently correct a date the user never had a chance to weigh in on, on a field the app itself had just set moments earlier. `test_auto_filled_date_is_preserved_when_a_later_detected_date_differs` exercises exactly this sequence.

6.3 Test results / verification method

This session's sandbox had no network access to install `fastapi`/`pytest`/`httpx` (none were already present), so unlike Checkpoints 3 and 5 – which eventually got a real `pytest` run, either in-session or in a later post-checkpoint pass – **the new backend tests in this checkpoint have not been executed via a real `pytest`/`TestClient` run at all.** In its place:

- All touched `.py` files were confirmed to pass `python3 -m py_compile` cleanly.
- `overrides_store.py`'s migration and `upsert_override` round-trip/clear/preserve behavior were verified by direct standalone execution against a real `sqlite3` connection (not mocks) – including a hand-built table simulating a pre-Checkpoint-6 `overrides.db` to confirm the migration path.
- The full auto-fill decision (`dossier.py`'s `assemble_dossier()` plus `api.py`'s auto-fill branch) was run end-to-end as a standalone script against a real confirmation-email text fixture on disk, through the real extraction pipeline (`extract.get_or_extract`, `date_extract.extract_application_date`) – confirming the detected date, tier, and resulting override row all come out as designed.

The frontend edit was similarly not checked with a real Babel/JSX parse (Checkpoint 4/5's method, via a locally-installed `@babel/core`, which isn't present in this sandbox). In its place: a full diff against the pre-checkpoint file confirmed every changed line matches the intended edits with nothing unintended touched, and a hand-rolled bracket-balance walk (skipping string/template/comment contents) confirmed the edited `DossierSection` function's braces close cleanly at the expected point, with the residual imbalance from the same walk over the whole script block shown to be identical before and after the edit (a known false-positive from JSX text containing apostrophes, e.g. "don't" – present equally in both versions, not something this edit introduced).

This checkpoint has not had a manual click-through against the real working tracker, unlike Checkpoint 5's post-checkpoint verification (§5.4). **A real `pytest` run and a manual click-through of the three date-applied render states (conflict / weak suggestion / provenance caption) are the first item in the Next Steps box** and should be treated as required before this checkpoint is considered fully verified, not optional follow-up.

6.4 Deliverable

`jobtracker-hub-item6-checkpoint06.zip` – full repository source (minus `.venv`, `.git`, caches, and local workspace state), plus an updated `CHANGES.md` at the repository root and this updated dev log.

6.5 Known limitations

- **Not yet verified by a real test run or manual click-through** – see the verification-method subsection above and the Next Steps box; this is the checkpoint's single most important open item, more so than any of the items below.
- Carries forward all Checkpoint 1–5 limitations unchanged (US-format phone regex, empty-password-only PDF decryption, no `.docx` support, trailing-boilerplate bleed into the last-matched role section, arbitrary-but-deterministic alphabetical job-posting tiebreak, no per-document source attribution on merged contacts, no non-US date formats, no signal for which confirmation document wins when a folder has more than one).